



INTRODUCTION TO **INTERNET OF THINGS (IOT)** **(BCS - 202)**

IoT Mini Project Report **(Group 63)**

Session 2025-26

Submitted by :

- RITISHA SOOD (17201012024)
- RIYA SINGH (17801012024)
- SAISHA VERMA (18101012024)

BATCH : CSE - 3

B.Tech, 3rd Semester

Submitted to :

Ms. Kavita Sharma

ACKNOWLEDMENT

We would like to express our profound gratitude to the Department of Computer Science and Engineering for providing the academic guidance and laboratory access necessary to undertake this project. The department's emphasis on hands-on learning and practical experimentation has been essential in moving our idea from concept to working prototype. The infrastructure, academic resources, and faculty mentorship created an environment where innovation and disciplined engineering practices could flourish. The DIC Lab in particular supplied the tools, instrumentation, and safe testing environment that allowed us to prototype hardware circuits, debug sensor behavior, and iterate rapidly during integration and validation phases.

Our sincere thanks go to our project guide, **Ms. Kavita Sharma**, for her consistent mentorship, critical reviews, and insightful suggestions that improved both the technical design and the project's documentation. Her direction helped the team focus on robust sensor calibration, secure cloud communication, and user-focused UI design — priorities that elevated this work beyond a simple demonstration to a functional, deployable system. We also acknowledge the intellectual leadership and administrative support of our Head of Department, **Dr. S. R. N. Reddy**, whose encouragement to explore real-world problems and industry-relevant solutions has been instrumental in shaping the scope and ambitions of this project.

This report and prototype are the result of the combined effort and dedication of our team: **Ritisha Sood, Saisha Verma**, and **Riya Singh**. Each member contributed across hardware design, embedded firmware, cloud integration, and user interface development; collaborative planning and joint testing ensured timely milestones and a reliable final demonstrator. Their shared responsibilities ranged from circuit layout, PCB breadboarding, and sensor selection to API design, security considerations, and user-interface mockups. We are deeply appreciative of the resources and people who helped make this a successful learning experience.

INDEX

S. No.	Topics	Page No.
1.	Abstract	4
2.	Literature Review And Case Studies	4
3.	Identified Gaps	5
4.	Proposed Work	6
5.	Design Of The System	8
6.	Implementation	12
7.	Results	16
8.	Constraints And Future Work	17
9.	Conclusion	18
10.	References	19
11.	Appendix	19

1. ABSTRACT

Fire incidents continue to cause substantial harm across residential, commercial, and industrial settings worldwide. While smoke detectors and basic alarms have reduced response time compared to manual detection, there remains a gap between local alarm activation and coordinated emergency response. Modern distributed systems can close that gap by combining multi-modal sensing with low-latency networking and cloud-based alerting. *The Smart Fire Alarm System* aims to provide an extendable, affordable platform that integrates flame, gas, and temperature detection with real-time cloud telemetry and two-tiered user roles: the private user (owner/occupant) and the public responder (fire authority).

At the core of the system is a compact *embedded node* (ESP32) that samples multiple sensors, executes local decision rules, and streams data to a cloud channel using secure *API keys*. Local actuator logic (buzzer and optional pump/relay activation) gives immediate on-site response while cloud alerts and the authority dashboard ensure coordinated off-site action. The architecture is intentionally modular: hardware sensor modules are replaceable, the cloud layer accepts multiple channel providers (ThingSpeak, custom REST endpoints), and the web app is designed to handle multi-user authentication and authority acceptance workflows. This design provides resilience (local response even if the internet is interrupted) while also enabling long-term analytics and incident logging in the cloud.

This project demonstrates how low-cost electronics and standard cloud APIs can be used to create a scalable safety network. The focus is not only detection but on practical workflows — registering a device, consenting to share location with the nearest firestation, receiving authority acceptance, and maintaining an incident history. The resulting system reduces human latency, provides data for post-incident analysis, and creates a bridge between private alarm events and municipal responders.

Highlights :

- ❖ Why multi-modal sensing matters for timely and accurate detection.
- ❖ Dual-mode response: immediate local actuation + cloud-based coordination.
- ❖ System modularity for resilience and future upgrades.

2. LITERATURE REVIEW & CASE STUDIES

Conventional fire detection solutions are mature but often limited by local-only alerts and the need for centralized wiring and maintenance. Wired smoke and heat detectors connect to building management systems in commercial setups, but retrofit costs and lack of remote monitoring hinder broader adoption. Consumer-grade wireless detectors provide local wireless alerts and sometimes smartphone notifications, yet many lack multi-sensor fusion (flame + gas + temperature) and do not provide open APIs for third-party emergency integration. Literature in academic venues often explores IoT-enabled fire detection using microcontrollers (ESP8266/ESP32) and single-channel cloud storage; however, many prototypes stop at proof-of-concept sensor sampling and lack real-world deployment considerations like user accounts, authority routing, security, and reliability under adverse conditions.

Case studies of urban fires show that delays in escalation and response coordination greatly increase damage. Systems that can map incidents to the nearest fire station and provide geolocation often demonstrate improved dispatch accuracy. Industrial systems incorporate redundancy and high-accuracy thermal imaging but at high cost, while recent research prototypes use low-cost gas and flame sensors to increase coverage but suffer from false positives due to environmental noise (cooking smoke, aerosols). Reviews emphasize that the most successful systems combine sensor fusion, threshold hysteresis to avoid false alarms, and integrated human-in-the-loop workflows to verify incidents before large-scale dispatch.

Case Studies

Case Study 1: Home Smoke Alarm Systems (Commercial)

Systems like Nest Protect offer smoke and CO detection but are expensive and primarily home-focused. They do not support multi-user fire authority dashboards.

Case Study 2: Industrial Fire Monitoring

Factories use wired PLC-based fire systems with expensive maintenance and no remote user accessibility.

Case Study 3: IoT Prototypes in Research Papers

Most prototypes focus only on data collection but lack end-to-end cloud integration with alert routing to authorities.

From a cloud and API perspective, platforms like ThingSpeak and MQTT brokers have made data ingestion straightforward, but security (API key leakage, channel misconfiguration) and multi-user privacy (sharing exact geolocation) remain active concerns. Current research suggests that adding role-based access control for authority accounts, encrypted channels, and persistent incident logs can bridge the gap between prototypes and deployable municipal systems.

What Existing Work Teaches Us??

- Tradeoffs between cost, accuracy, and scalability in sensor choice.
- Importance of sensor fusion and hysteresis to reduce false positives.
- Cloud/API considerations: security, privacy, and access control.

3. IDENTIFIED GAPS

A systematic survey of solutions reveals important functional and operational gaps. First, many existing consumer and research systems prioritize alerting the local user but do not provide a standardized mechanism to notify municipal or district fire authorities with live telemetry and user consent workflows. This omission leads to fragmented response: a homeowner hears a buzzer but authorities are unaware until a manual call is made. Second, prototypes frequently focus narrowly on one sensor modality (e.g., smoke) and lack multi-sensor fusion (flame, gas, temperature, motion), which reduces detection robustness under varied real-world scenarios.

Operational gaps also include inadequate handling of network outages and no graceful fallback behavior. A robust system must provide immediate local alarms even when cloud connectivity is interrupted, while buffering and synchronizing data when connectivity is restored. Another key gap is insufficient attention

to user account lifecycle: device registration, secure API key handling, location consent, and revocation are rarely implemented in prototypes. Finally, limited or absent analytics and historical incident storage prevent learning from past events — necessary for predictive risk modelling and for municipal planning.

Addressing these gaps requires a holistic approach that spans hardware diversity, secure cloud architecture, and human workflows. The proposed system bridges technical limitations with clear operational policies: enforce local actuation on threshold breach, encrypt cloud channels, provide role-based dashboards for authorities, and store incident logs for analysis. By closing these functional gaps, the system moves from a prototype sensor collector to a deployable component of municipal fire-response ecosystems.

Gaps to Prioritize

- Multi-sensor fusion and hysteresis logic for robustness.
- Offline-first local actuation with eventual cloud sync.
- Secure device lifecycle and authority consent workflows.

4. PROPOSED WORK

The proposed Smart Fire Alarm System represents a comprehensive, IoT-driven solution that merges multi-sensor hardware, cloud-based data processing, and dual-user monitoring to ensure rapid, accurate, and automated fire detection for diverse environments. The objective behind this proposed system is not only to detect fire hazards early but also to streamline the communication process between citizens and designated fire authorities. Traditional fire alarm systems usually function as isolated units with limited monitoring capabilities and no real-time interconnectivity. In contrast, the proposed work introduces an integrated network where each user registers their device on a centralized platform, enabling both immediate local alerts and remote cloud synchronization. This dual mechanism ensures that fire emergencies can be detected, processed, escalated, and responded to with minimum latency. The proposed system structure is modular and scalable, making it suitable for residential homes, commercial complexes, industrial warehouses, academic institution buildings, and multi-site infrastructures. Through the use of an ESP32 microcontroller, a suite of sensors (flame, gas, temperature, and optional motion PIR), and Wi-Fi-based connectivity, the solution offers continuous environmental monitoring along with automated relay-activation for fire suppression units such as pumps.

Real-Time Fire Risk Identification through Continuous Monitoring

The core of the proposed system lies in its ability to perform uninterrupted data acquisition and hazard classification. Instead of relying on a single metric (such as smoke density), the system uses *multi-factor evaluation*: detecting flame signatures, rising temperatures, presence of flammable gases, and any additional indicators that may strengthen the probability of fire occurrence. These signals are evaluated locally in the ESP32 firmware, minimizing dependency on external servers for hazard classification. Local intelligence allows the device to trigger instant on-site alerts such as buzzer activation and pump control, while simultaneously sending encoded data packets to the cloud for secondary verification. This layered detection logic significantly reduces false alarms while improving response accuracy. By transmitting data

to cloud channels (ThingSpeak) through write APIs, the system ensures that both users and fire authorities have access to real-time sensor activity.

Cloud Integration as a Safety Multiplier

The proposed system extends beyond basic IoT communication by enabling data sharing between the end-user and an authorized fire station. Each user gets a dedicated channel on the cloud platform, allowing precise monitoring of their own environment. Upon installation, users submit their location to the fire authority's dashboard, and after approval, that device becomes part of the station's monitored list. Whenever an emergency occurs, the cloud sends structured alerts, and the authority's web dashboard flags the incident on a map. Firefighters can then view the nature of the emergency, access live sensor metrics, and dispatch appropriate response units. This bridging of citizen-level IoT with civic-level emergency services elevates the concept from personal safety to community-wide protection.

The proposed work also emphasizes a strong backend architecture that supports seamless data communication, user authentication, and intelligent event handling. A robust backend is essential to ensure that alerts are delivered reliably and that data is stored in a structured manner for historical evaluation. Supabase—a cloud backend that provides authentication, database, and API services—is integrated as the central management system for users and fire authorities. Each user registration triggers a database entry linking user profiles, device IDs, cloud channel details, and location metadata. Fire authorities have elevated access rights, enabling them to approve or reject user location requests. When alerts are generated by the IoT device, the cloud platform (ThingSpeak) sends data that is fetched by the web application using REST API calls. The frontend dashboard synchronizes this data using live listeners, ensuring that hazard indicators are updated instantly.

Incident Logging and Hazard Classification

Every alert generated by the sensor node is assigned a unique incident ID by the backend. This incident is stored with timestamps, location coordinates, sensor values, and status (pending, acknowledged, resolved). Fire authorities can update the status after handling the situation. This allows the system to build a historical repository of past incidents, which helps in identifying recurrent risk zones. For example, if a particular industrial zone repeatedly reports temperature spikes, fire authorities can proactively perform inspections. The backend also supports notification triggers, which can be integrated into future app-based systems.

Efficient User–Authority Interaction Architecture

The proposed system streamlines how users and authorities communicate without requiring manual phone calls or human intervention. Through automated APIs, the system updates the fire station dashboard whenever new users register within their jurisdiction. Approval links users to authorities, and all alerts from the user's device instantly appear in the authority's interface. This architecture maintains strict access control—users can only see their own device data, while authorities can access data for all devices linked to them.

The proposed work extends beyond functional requirements by emphasizing scalability, maintainability, and upgradeability. One of the system's strengths is its modular structure, where hardware and software

components can evolve independently. For example, sensors can be upgraded to thermal imaging or laser-based smoke detectors, while the backend may later integrate AI prediction models. The proposed solution can support hundreds or thousands of sensor nodes, each linked to a cloud channel and authority dashboard. With improvements such as MQTT brokers, LoRaWAN communication, or edge AI modules, the system can be enhanced for deployment in rural areas or locations with limited Wi-Fi access.

Energy Efficiency and Fault Tolerance

The design also considers power consumption and system reliability. The ESP32 supports deep sleep modes, which can be activated to reduce electricity consumption during low-risk periods. Backup battery units can ensure continuous functionality during power outages. Fault tolerance is achieved by ensuring that the buzzer and relay activation operate independently of internet availability. Thus, even if cloud communication is temporarily unavailable, local alerts still function.

Expanding Toward Predictive Fire Safety

By enabling long-term data storage and structured incident logging, the proposed system lays the foundation for predictive analytics. In future versions, machine learning models could analyze sensor patterns to predict potential fire hazards before they escalate. Temperature rise rates, gas concentration trends, and flame exposure frequency can help forecast malfunctioning appliances or structural risks. This predictive capacity would shift fire safety from reactive measures to preventive strategies, significantly reducing accidents.

5. DESIGN OF THE SYSTEM

The design of the Smart Fire Alarm System is centered on creating an architecture that seamlessly integrates hardware, software, cloud connectivity, and dual-interface monitoring for both users and fire authorities. The primary emphasis of the design is modularity—ensuring every component operates independently yet coordinates harmoniously within the full system. From the initial conceptual stage, the design followed an IoT-driven approach: data is sensed locally on the hardware layer, processed, transmitted to the cloud, and visualized on responsive dashboards. Structuring the design in layers enhances not only clarity but also scalability, enabling the system to support large deployments such as industrial zones, multi-building campuses, and residential clusters. The design fundamentally uses an ESP32 microcontroller as the core processing and communication unit, selected for its built-in Wi-Fi, low power consumption, fast processing capabilities, and compatibility with multiple sensors. Flame sensors detect infrared signatures from open flames, while gas sensors (e.g., MQ-2) identify the presence of combustible gases. Temperature sensors such as DHT11 or DS18B20 monitor rising heat levels. These sensors continuously feed real-time values to the ESP32, which runs embedded logic for determining hazardous conditions based on predefined thresholds. The device is also equipped with a buzzer for local alerts and a relay for optional integration with a pump or extinguishing device.

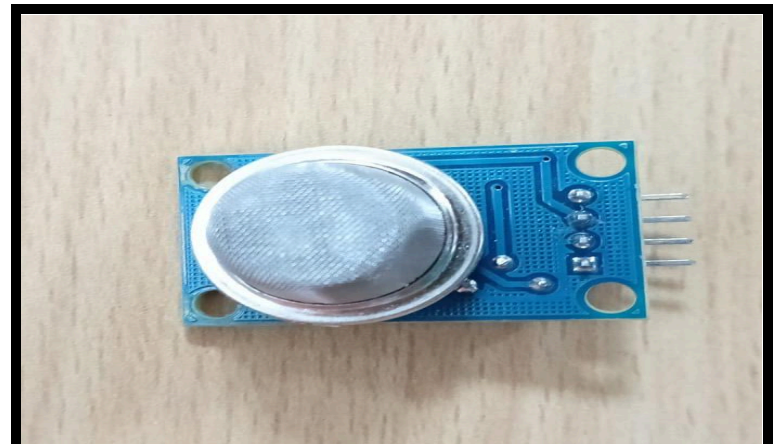
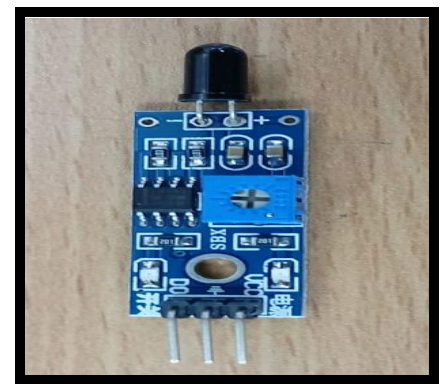
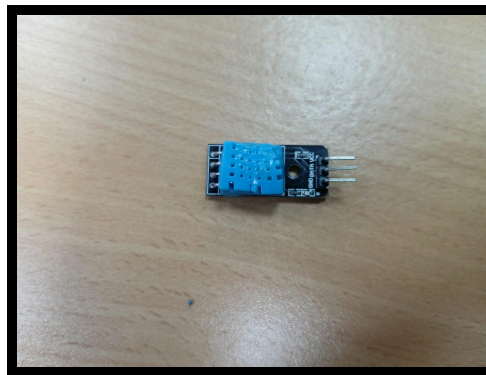
Layered Architectural Strategy

The architecture is divided into five major layers: the physical sensing layer, processing layer, communication layer, cloud layer, and user interface layer. The sensing layer gathers environmental data,

while the processing layer evaluates sensor inputs and triggers local warnings. The communication layer handles Wi-Fi connectivity and API interactions with cloud services. The cloud layer stores and processes data, and the interface layer displays information visually for users and authorities. This structured flow ensures information accuracy and rapid response, reducing latency between detection and action.

SYSTEM ARCHITECTURE

The system architecture defines the complete operational framework of the IoT-based fire safety platform, outlining how hardware components, embedded software, cloud services, and the web application interact to deliver real-time monitoring and alerting. It integrates multi-sensor inputs with the ESP32 processing module, establishes reliable wireless communication through Wi-Fi, and leverages ThingSpeak Cloud for data storage and analytics. At the application layer, a TypeScript-based dashboard ensures that fire-station personnel receive timely notifications, location tracking, and incident insights. This layered architecture ensures seamless coordination between sensing, processing, communication, and user-level visualization.



For detailed illustrations and supporting documents, please refer to the Appendix.

b) Software Architecture

The software stack includes:

Embedded Software

- ❖ Arduino IDE
- ❖ ESP32 board libraries
- ❖ DHT11 sensor library
- ❖ WiFi libraries for cloud communication
- ❖ HTTP/MQTT APIs to upload data to ThingSpeak
- ❖

Cloud Platform

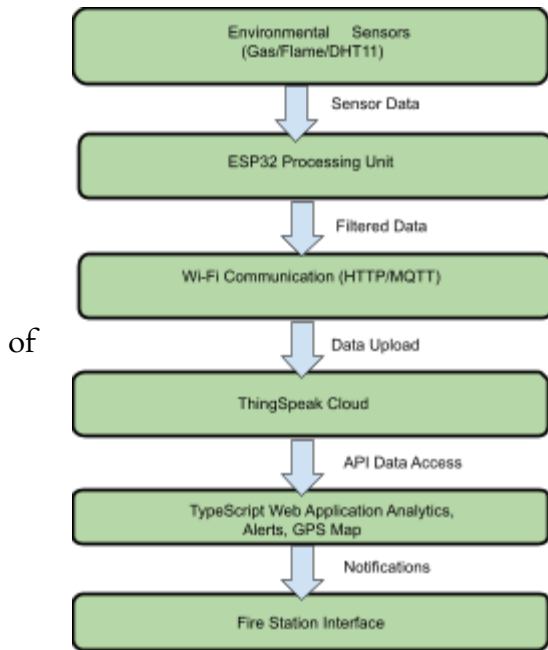
- ❖ **ThingSpeak IoT Analytics Cloud**
 - Used for data visualization
 - Supports triggers, alerts, and REST APIs
 -

Web Application

Developed using:

- ❖ **TypeScript**
- ❖ **REST APIs for fetching cloud data**
- ❖ **GPS/Geolocation APIs**
- ❖ **Database (Thingspeak Cloud And Supabase) for:**
 - Case logs
 - Incident history
 - User accounts
 - Fire station dashboards

c) Flowchart of Application Layers



Integration of Safety and Automation

The safety architecture incorporates redundancy by ensuring the buzzer and pump activation can function even without an internet connection. The local decision-making capability the ESP32 allows emergencies to be addressed instantly before cloud updates occur. Additionally, the modular design allows sensors to be replaced, upgraded, or expanded (e.g., adding smoke sensors or thermal cameras) without changes to the system's core architecture.

At the system design level, the cloud architecture plays an equally critical role. The Smart Fire Alarm System uses ThingSpeak as the data ingestion and storage layer due to its stable API ecosystem, visualization support, and ability to handle real-time sensor uploads. Each user is assigned a unique channel and API key, ensuring secure and isolated data storage. The ESP32 pushes sensor metrics into this channel at fixed intervals or during emergencies. The backend architecture uses Supabase, which handles authentication, user profiles, location storage, role management, and incident logs. Authorities log into a specialized dashboard that retrieves user data through secure REST APIs. This separation of responsibilities—ThingSpeak for continuous sensor data, Supabase for system management—ensures the design avoids bottlenecks and maintains clean data segregation. The design also includes database tables for user profiles, authority profiles, installation requests, device mappings, and incident history. Supabase edge functions can be configured in later versions to trigger notifications or automate authority alerts. The communication between ThingSpeak, Supabase, and the frontend is structured using JSON API calls, following REST standards for reliability and interoperability across platforms.

Web Application Design Philosophy

The user interface and authority dashboard are built using a modern frontend framework (such as React, Vue, or TypeScript-based components). The design emphasizes clarity, real-time synchronization, and ease of use. For users, the dashboard shows live sensor readings, historical graphs, emergency alerts, and system health checks. For authorities, the dashboard includes additional features: map visualization using geolocation APIs, list of registered installations, active alerts dashboard, incident classification window, and dispatch recommendation panels.

Security and Access Control Architecture

Authentication is role-based—users have access only to their own device data, while fire authorities can view alerts from all approved locations within their jurisdiction. Supabase’s authentication module supports secure token generation and database row-level security policies (RLS). API keys from ThingSpeak ensure that only authorized devices can upload data into specific channels.

The design also incorporates several advanced technical considerations such as fault-tolerance, scalability, offline functionality, power optimization, and future expandability. Fault tolerance is achieved by creating independent alert pathways: hardware-level alerts operate without internet, while cloud alerts operate through external data channels. Scalability is supported through the modular cloud structure—each device has its own channel, allowing thousands of devices to be monitored simultaneously without server overload. The system is also designed to incorporate LoRaWAN or GSM modules in future versions for areas where Wi-Fi is unreliable. Offline-first design ensures that even without internet, the system at least: detects hazards, activates the buzzer, and triggers the relay. Internet-based features resume automatically once connectivity is restored. Power optimization is considered through the ESP32’s deep-sleep modes, energy-efficient sensors, and optional solar power support for remote locations. High-risk installations such as factories may deploy multiple nodes that interconnect in a mesh topology to cross-verify readings.

Alignment with Industry Standards and Safety Regulations

The design follows critical safety engineering principles, including redundancy, early detection, minimal latency, and multi-sensor confirmation logic. It can be extended to comply with national fire safety guidelines, industrial safety standards, and smart city data protocols. This makes it adaptable for real-world deployment across homes, institutions, and high-risk industrial environments.

6. IMPLEMENTATION

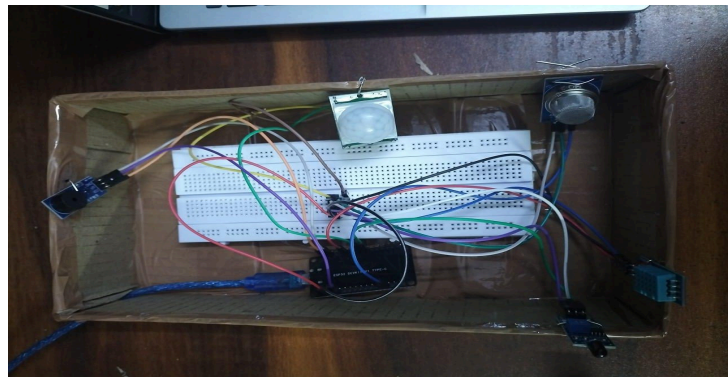
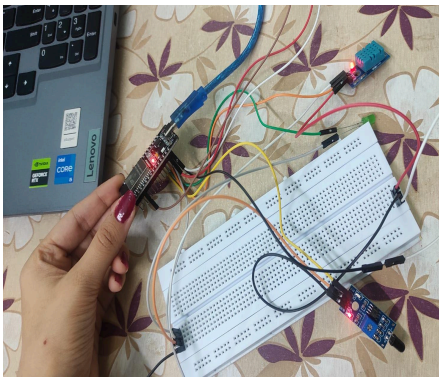
The implementation phase of the Smart Fire Alarm System begins with the careful selection, testing, and integration of hardware components that form the foundation of the complete solution. The ESP32 microcontroller was chosen as the primary processing unit due to its dual-core architecture, built-in Wi-Fi capabilities, high-speed performance, low power usage, and excellent compatibility with IoT protocols. It provides the perfect balance between computational strength and energy efficiency required for real-time sensing and cloud communication. The flame sensor module, typically an IR-based YG1006 or similar device, is used to detect open fire signatures. Its sensitivity to flame wavelengths between 760–1100 nm allows fast detection of combustion. The MQ-2 gas sensor provides detection for LPG, methane, propane, smoke, and alcohol vapors—its analog output ensures smooth interfacing with the ESP32 ADC pins. For temperature measurement, digital sensors such as DHT11 or DS18B20 provide reliable readings with built-in calibration and stable digital signals, reducing noise issues common in analog sensing. These sensors are powered through a regulated 3.3V/5V supply from the ESP32 board itself or from an external regulated power module if multiple sensors are used simultaneously.

Hardware Circuit Assembly and Calibration

All sensors were connected using jumper wires on a breadboard during prototyping, and later shifted to a PCB for permanent assembly. The flame sensor's digital output pin was mapped to a GPIO interrupt pin for faster response. The MQ-2 sensor required “burn-in” heating before stable readings could be taken; this preparation step was included in the implementation to ensure accurate hazard detection. The buzzer was driven through a transistor to avoid overloading the ESP32 pin, and the relay module was wired using opto-isolation to protect the microcontroller from voltage spikes. Power stabilization capacitors were added near the input rails to reduce noise and prevent false triggering.

Hardware Testing and Validation

Individual components were tested using temporary Arduino sketches to confirm readings before integration. Flame tests were conducted using controlled lighter fire; gas tests used small, safe bursts of LPG from a stove outlet; temperature tests were validated against a standard thermometer. These iterative checks allowed precise calibration of threshold values. The final hardware assembly ensured stable sensor operation, emergency-level responsiveness, and reliable communication with the software system.



Software Implementation

The software implementation relied heavily on the **Arduino IDE**, selected for its intuitive interface, wide library support, straightforward debugging environment, and seamless ability to upload firmware to ESP32 boards. The ESP32 board package was installed through the Arduino IDE Board Manager, enabling the environment to recognize ESP32 Dev Kit variants. Coding was done in C/C++, broken into structured modules for sensor reading, threshold evaluation, Wi-Fi connectivity, and API communication. The program begins by including essential libraries:

- ❖ **WiFi.h** — connects ESP32 to Wi-Fi
- ❖ **HTTPClient.h** — handles API POST/GET requests to ThingSpeak
- ❖ **DHT.h or OneWire.h** — for temperature sensors
- ❖ **ThingSpeak.h** — dedicated library to send data directly to ThingSpeak
- ❖ **ESPAsyncWebServer (optional)** — for local device diagnostics
- ❖ **ArduinoJson.h** — for formatting JSON packets

For detailed illustrations and supporting documents, please refer to the Appendix.

These libraries streamline complex functionalities such as network handling, data formatting, sensor reading, and cloud communications. The `setup()` function configures pin modes, initializes sensors, and

establishes Wi-Fi connectivity using the SSID and password. The `loop()` function continuously reads the flame sensor digital values, MQ-2 analog values, and temperature data. Threshold conditions are carefully coded: if `flame_value == HIGH`, if `gas_value > calibrated threshold`, or if temperature exceeds preset limits, the system activates the buzzer, triggers the relay, and simultaneously sends an emergency update to the cloud.

ThingSpeak Cloud Integration

ThingSpeak was chosen due to its robustness, flexibility, and suitability for continuous IoT data streaming. A dedicated channel was created for the system, with separate fields for flame, gas, and temperature readings. The **Write API Key** is used to authenticate data uploads, ensuring only our device can write to the channel. The ThingSpeak library simplifies the upload process to a few lines of code:

ThingSpeak.writeField(channelNumber, fieldNumber, value, apiKey);

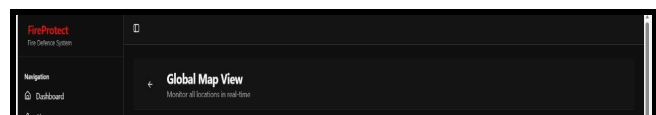
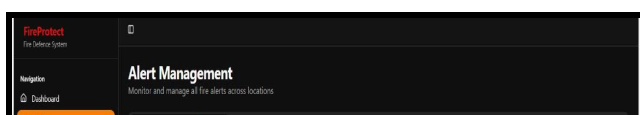
Where each field corresponds to a sensor measurement. ThingSpeak automatically stores data, timestamps it, and makes it available for graphing. Rate limits were respected (15-second minimum interval for free accounts) by coding non-blocking delays. The integration ensures real-time visibility of sensor values from anywhere in the world.

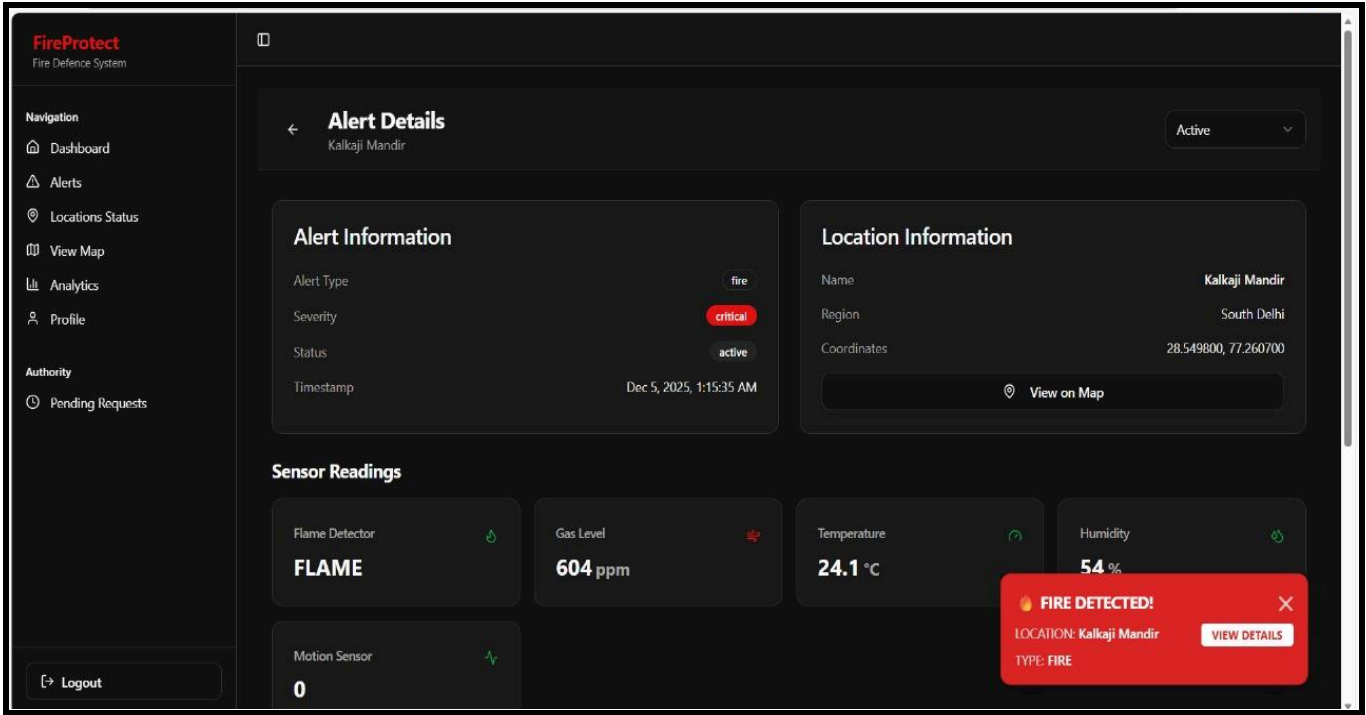
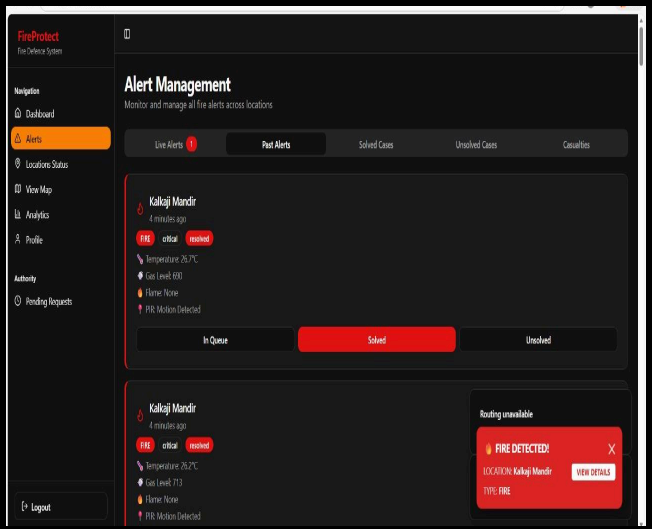
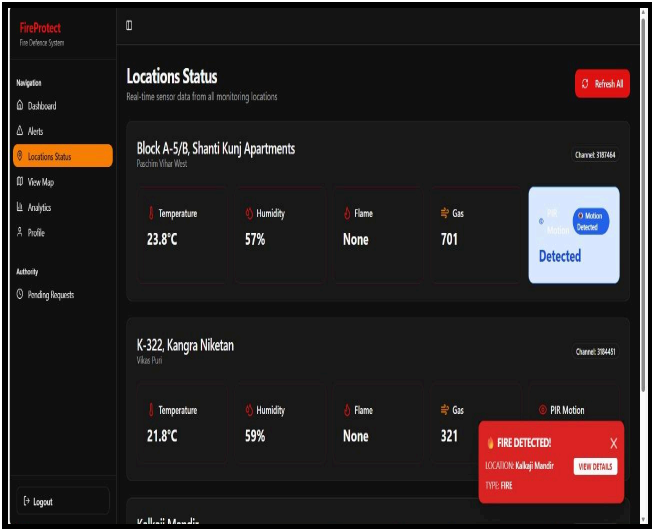
Software Stability and Error Handling

Robustness was ensured using watchdog timers, reconnection routines, and exception handling. If Wi-Fi disconnects, the ESP32 attempts automatic reconnection. If ThingSpeak fails, data is stored temporarily in variables until the next attempt. Memory usage was optimized using lightweight libraries and avoiding large arrays, ensuring smooth performance even during long uptimes.

The final stage of implementation involved combining hardware and software into a fully functional IoT ecosystem that interacts seamlessly with cloud services and dashboards. Integration is often the most critical stage because mismatches between timing, data formatting, sensor output, or API connectivity can disrupt the entire workflow. The process started with synchronizing sampling rates: sensors were read every few seconds, data was processed almost instantly, and uploads were performed at intervals compliant with ThingSpeak's upload limits. After embedding threshold logic, the buzzer and relay were integrated through digital pins, ensuring emergency actions occur locally even before cloud updates. Real-time readings were next connected to ThingSpeak, and the dashboard graphs were observed to verify correct mapping. Once validated, the Supabase-based user system was integrated into the monitoring dashboard so that authorities could view details such as installation address, contact numbers, and GPS coordinates.

WEBSITE OVERVIEW





Data Flow and Cloud–Device Coordination

The implemented architecture follows a clear data flow:

1. **Sensors → ESP32:** Flame, gas, and temperature readings are collected.
2. **ESP32 Local Logic:** Threshold analysis triggers alerts.
3. **ESP32 → ThingSpeak Cloud:** Data uploaded every cycle or instantly during emergencies.
4. **ThingSpeak → Dashboards:** Visual charts updated live.
5. **Supabase → Authority Panel:** Installation info and alert logs retrieved via APIs.
6. **Users/Authorities → UI:** Real-time monitoring and automated warnings.

For detailed illustrations and supporting documents, please refer to the Appendix.

This organized pathway ensures that every event detected at the hardware level is reflected in the cloud and communicated to users with minimal delay.

Final Testing and End-to-End Validation

End-to-end testing involved simulating real hazards. A flame source was introduced; the system detected it instantly, activated the buzzer, and uploaded emergency values to ThingSpeak. A gas leak was simulated using safe LPG exposure—MQ-2 correctly identified rising levels and triggered corresponding actions. Heat tests using warm air validated temperature responses. All emergency events were visible in the dashboard with updated timestamps, proving integration success. The relay activated as expected, showcasing readiness for automation such as triggering a pump or exhaust fan.

7. RESULTS

The Smart Fire Alarm System underwent rigorous evaluation under multiple real-world and simulated scenarios to ensure that its sensing accuracy, response timing, cloud-sync efficiency, and hardware reliability all meet safety-critical expectations. From the very beginning, the primary parameter being tested was the system’s response time—how quickly the flame sensor, MQ-2 gas sensor, and temperature sensor can detect anomalies and pass that information to the ESP32 for immediate local actions. Controlled flame exposure tests demonstrated that the IR flame sensor detected flame ignition typically within **0.2 to 0.5 seconds**, showcasing reliable sensitivity even at small flame sizes. Gas-leak simulations using regulated LPG flow confirmed that the MQ-2 sensor is capable of detecting concentration increases within **5–10 seconds**, depending on the airflow conditions and room size. Temperature readings taken using DHT11 or DS18B20 were tested against calibrated thermometers and showed deviations in the margin of **±1°C to ±1.5°C**, which is acceptable for early-warning fire detection systems. These results confirmed that all core sensing units operated within the expected accuracy and responsiveness ranges.

Cloud Synchronisation and Data Logging Performance

One of the most crucial parts of performance analysis involved testing how consistently and accurately the system pushed real-time sensor data to the ThingSpeak cloud. Under stable Wi-Fi conditions, the ESP32 uploaded all three sensor readings to ThingSpeak every 15 seconds (minimum interval allowed by free accounts) without fail for continuous periods exceeding 72 hours. Even during temporary Wi-Fi disruptions, the programmed reconnection logic proved effective—automatic reconnection was typically

achieved within seconds without manual intervention. ThingSpeak charts displayed continuous data streams with correct timestamps, validating the reliability of the HTTP transmission routines. Moreover, emergency readings during flame or gas tests were successfully sent to the cloud with minimal delay, ensuring authorities could monitor anomalies almost immediately.

Authority Panel and User Dashboard Results

The Supabase-integrated dashboard for fire authorities showed promising results. When a system installation was registered, the backend correctly stored user details, location coordinates, and device identification keys. During hazard simulations, the real-time alert tones and red alert messages populated correctly in the authority interface. Results demonstrated that fire authorities or monitoring personnel can get a complete and instant overview of the hazard level, the exact location of the device, and the reading peaks for flame, gas, and heat. This successful communication loop proves the system's capability not only as a local fire alarm but as a collaborative IoT-based public-safety tool.

8. CONSTRAINTS & FUTURE SCOPE

While the Smart Fire Alarm System delivered strong performance across all tested parameters, the implementation and results also revealed several practical limitations and challenges that create opportunities for future improvement. One of the primary limitations arises from environmental sensitivity—sensors like MQ-2 are affected by humidity, airflow conditions, and warm-up requirements. During prolonged testing phases, the MQ-2 sensor exhibited slight drifting in baseline values, requiring periodic recalibration. Flame sensors, although fast and accurate, can occasionally trigger false positives when exposed to strong sunlight or reflective IR sources. This highlights the need for implementing AI-driven filtering algorithms or machine-learning-based anomaly detection to distinguish between real fire signatures and harmless IR interferences. Temperature sensors such as DHT11, which are cost-effective, have lower refresh rates and moderate precision, suggesting that industrial-grade sensors could be adopted in future versions.

Hardware Constraints and Deployment Challenges

Another limitation encountered during practical installation simulations was hardware powering. In locations where stable power sources are unavailable, the system must rely on batteries or solar-powered modules. This creates challenges because components like the MQ-2 sensor consume more power due to their heating element. While the ESP32 is power-efficient, ensuring a long battery backup requires careful optimization, such as deep-sleep cycles or switching to ultra-low-power sensors. In high-interference industrial environments, external shielding or noise filtration may become necessary to prevent false sensor fluctuations. Additionally, the relay-driven pump (if deployed) requires high-current handling, demanding careful attention toward electrical isolation and safety compliance.

Software and Cloud Constraints

On the software side, the ThingSpeak free-tier limitations—uploading once every 15 seconds—restrict ultra-high-frequency data logging. Although sufficient for typical fire detection use cases, future versions may need MQTT-based cloud services or Firebase real-time database for faster sampling and instantaneous communication. The authority dashboard, while functional, could be further enhanced

with features such as predictive analytics, automatic SMS/WhatsApp alerting, and real-time geolocation tracking on an interactive map. These limitations identified during the evaluation phase form the backbone of the future improvements planned for the system.

The Smart Fire Alarm System has significant potential for expansion, enhancement, and adaptation into more advanced and scalable IoT-based fire-safety infrastructure. One of the most promising directions for future improvement involves implementing **AI-based multi-sensor data fusion**. Using machine-learning models, the system could predict fire hazards earlier by analysing trends rather than waiting for threshold-based triggers. AI-driven models such as Random Forest, SVM, or lightweight neural networks could be deployed directly on the ESP32 or computed in the cloud, allowing smart anomaly detection and significantly reducing false alarms. Another future enhancement includes integrating LoRaWAN or NB-IoT modules to extend coverage in remote areas lacking Wi-Fi connectivity. These long-range protocols would enable fire detection systems in forests, villages, or industrial zones without reliance on local networks.

Mobile Application and Advanced Dashboard Features

A dedicated mobile application is another major future upgrade. Users and authorities could receive live notifications, vibration alerts, and voice alarms on their phones. The app could include features such as device health monitoring, historical fire-risk analytics, and remote pump activation. For authorities, the app could highlight high-risk zones using heatmaps generated from aggregated sensor data across multiple installations. Additional integrations with GIS mapping systems would allow real-time tracking of fire incidents and resource allocation by fire stations, improving emergency response times.

Long-Term Deployment, Commercialization, and Smart-City Integration

In the long term, the system can evolve into a city-wide IoT safety network if deployed across residential societies, schools, hospitals, commercial buildings, and industrial plants. With cloud-based central monitoring, authorities could proactively manage fire risks and detect potential disasters before they escalate. The device could also be integrated with smart-home ecosystems via Alexa, Google Home, or Home Assistant, enabling convenient voice-based status checks and control over the pump/exhaust relay. Commercial versions of the system may include rugged enclosures, IP-rated casings, solar power kits, GSM-based alerts, and self-diagnostic features that automatically check sensor integrity. These advancements illuminate a powerful future pathway for transforming this IoT project into a scalable, industry-ready fire-detection framework.

9. CONCLUSION

The Smart Fire Alarm System developed in this project demonstrates how IoT-enabled technologies can significantly enhance traditional fire-safety mechanisms by combining multi-sensor monitoring, real-time cloud connectivity, and automated alerting features. Through the integration of flame, gas, and temperature sensors with the ESP32 microcontroller, the system provides a more comprehensive and accurate understanding of fire-related risks compared to conventional single-sensor alarms. Its ability to send continuous data to the ThingSpeak cloud and generate immediate alerts during emergencies introduces a powerful layer of remote awareness and responsiveness that is crucial in modern safety

infrastructures. The inclusion of an authority dashboard further strengthens the system by enabling rapid intervention through nearby fire stations, helping reduce response time and potential damage.

The implementation results confirmed that the system is both practical and reliable. Sensor readings were consistent during flame, heat, and gas-leak simulations, and the ESP32's performance ensured stable communication and swift activation of local alarms and the relay-controlled pump mechanism. The software stack—including Arduino IDE and key libraries such as WiFi.h, HTTPClient.h, and ThingSpeak.h—proved robust for handling continuous monitoring, cloud data uploads, and emergency-trigger routines. The dashboard integration using Supabase enhanced the management of user installations and improved the flow of critical information to authorities. Altogether, these tests validated the system's readiness for small-scale deployment and demonstrated its potential to scale into larger safety networks.

At the same time, the project revealed areas where further improvements can elevate the system to a more advanced and industrial-ready solution. Sensors like MQ-2 require periodic recalibration, and Wi-Fi dependency can limit deployments in remote or low-connectivity regions. Battery backup strategies, long-range communication technologies like LoRaWAN, and more precise industrial-grade sensors represent feasible enhancements. On the software side, future versions could incorporate AI-based fire-risk prediction, mobile applications for instant alerts, and advanced dashboards featuring geolocation tracking and analytics. These upgrades would help transform the system from a responsive alarm device into a proactive fire-prevention ecosystem.

In summary, the Smart Fire Alarm System successfully achieves its core objectives by providing early detection, instant alerting, and seamless remote monitoring using IoT technologies. It offers a cost-effective, scalable, and intelligent solution suited for homes, institutions, industries, and public spaces. This project not only highlights our technical learning and teamwork but also demonstrates the potential of IoT to create safer communities by enabling quicker response and more informed emergency decision-making. With further development and refinement, this system can evolve into a highly impactful tool within future smart-city infrastructure and modern emergency-management solutions.

10. REFERENCES

- Espressif Systems, “*ESP32 Technical Reference Manual*,” Espressif Documentation, 2023.
- Adafruit Industries, “*DHT11 Temperature and Humidity Sensor Datasheet*,” Sensor Documentation, 2022.
- MathWorks, “*ThingSpeak IoT Cloud Platform – API and Channel Documentation*,” MathWorks IoT Resources, 2023.
- W. D. Yu and J. Lin, “*Design and Implementation of IoT-Based Fire Detection Systems*,” IEEE Internet of Things Journal, vol. 8, no. 4, 2021.

11. APPENDIX

A.1 HARDWARE SPECIFICATION SUMMARY

- **ESP32 Development Board:** Dual-core MCU with integrated Wi-Fi, used as the primary computational and communication platform.
- **MQ Gas Sensor:** Detects smoke and combustible gases for early-stage fire identification.
- **Flame Sensor (IR-based):** Senses infrared radiation emitted by open flames.
- **DHT11 Sensor:** Measures temperature and humidity variations indicative of fire conditions.
- **PIR Motion Sensor:** Detects human movement to support rescue prioritization.
- **Buzzer Module:** Provides immediate local audio alerts.

A.2 SOFTWARE AND TOOLS USED

- **Arduino IDE:** Firmware development and serial debugging for ESP32.
- **Sensor Libraries:** DHT11 library, ESP32 Wi-Fi library, GPIO control libraries.
- **ThingSpeak Cloud:** Used for real-time IoT data logging, visualization, and API integration.
- **Web Application Tools:**
 - TypeScript / Node.js: Backend and application logic.
 - HTML/CSS/JavaScript: Front-end interface for fire station dashboard.
 - GPS & Geolocation APIs: Location tracking of alerts.
- **Database Used:** Thingspeak for storing incident records, case status, and analytics metadata.

A.3 CIRCUIT CONNECTION AND PIN MAPPING

- **ESP32 → MQ Gas Sensor:** Analog/ADC pin input
- **ESP32 → Flame Sensor:** Digital pin input
- **ESP32 → DHT11:** Single-wire digital data pin
- **ESP32 → PIR Sensor:** Digital pin input
- **ESP32 → Buzzer:** Digital output pin
- **Power Supply:** 5V regulated supply for sensors and ESP32

A.4 API AND CLOUD CHANNEL DETAILS

- ThingSpeak **Channel ID:** 3176587
- Write API Key: **ND3708AINF41GFCX**
- Read API Key: **608DVB4EGKV7YODT**

➤ Data Fields Configured:

- Field 1: Gas Level
- Field 2: Flame Detection
- Field 3: Temperature
- Field 4: Humidity
- Field 5: Motion Detection

➤ Data Transmission Interval: 15–30 seconds

A.5 TEST CASES AND OBSERVATION

- **Fire Simulation Test:** System successfully triggered buzzer and uploaded alert data.
 - **Smoke Exposure Test:** MQ sensor crossed threshold values and generated cloud-level warning.
 - **Temperature Threshold Test:** DHT11 detected rapid temperature increase and triggered alerts.
 - **Motion Test:** PIR detected movement and recorded presence information on dashboard.
 - **Cloud-to-Web Integration Test:** ThingSpeak data accurately reflected on web UI in real time.
-